

土木工程規範問答系統（RAG 雙軌對照版）

以檢索增強生成（RAG）為核心的土木規範 PDF 智慧問答系統
GitHub 專案連結：<https://github.com/ching-pochiu/rag>

一、專題動機

土木工程規範文件（CNS 標準、施工規範等）動輒數百頁，內含大量表格化數據（鋼筋牌號、材料容許差、強度規定等），工程人員在審查或施工時常需要「翻頁找數字」，效率低且容易漏看。本專題以 RAG 架構讓使用者用自然語言提問即可快速定位正確條文與數值，並透過多層機制降低 LLM 生成答案時「編造數字」的風險——這對規範查詢這種容錯率極低的場景尤其重要。

與原型 ConSpec-Agent 的差異

本專題最初源自筆者在團隊專題 ConSpec-Agent（工程品質審查系統，GitHub: [ching-pochiu/conspec-agent](https://github.com/ching-pochiu/conspec-agent)）中負責的「CNS 法規問答」模組。回頭對照兩邊實際的程式碼後，發現原模組其實已經有 Cross-Encoder reranker 把關排序品質（甚至用了比這個專題更大的模型），但在另外幾個地方仍有明確可以強化的空間：切割時對表格與條文結構視而不見、檢索只有向量一條軌、答案生成後完全沒有事實查核。因此把這部分獨立拉出來重新設計，把切割方式、檢索管道、生成後查核這三層都補上結構化與驗證機制：

項目	ConSpec-Agent 原模組	這個獨立專題
生成後查核	無：檢索候選直接組成context餵給Gemini，答案內容沒有事後驗證機制	Groundedness 檢查 + 引用頁碼反查——兩邊都有reranker的情況下，這是唯一「連自己生成的答案都會回頭查」的一層
切割方式	兩軌並行：①23筆由Gemini一次性協助抽取的精選條文JSON（數量固定，非通用規則）；②全文OCR文字依頁切分後用「累加至900字上限」的貪婪合併切塊，只在行邊界斷開，對表格/節號/子項等結構完全無感知	條文依節號/子項邊界切割；表格另外偵測欄位結構，依牌號/代號分組逐塊切割——都是通用規則，可套用到任何新上傳的PDF
檢索方式	純向量檢索（Ollama qwen3-embedding:4b + ChromaDB cosine相似度），兩個collection分別查詢後直接合併候選池，沒有關鍵字比對這條軌	向量 + BM25 + RRF 融合排序：兩種訊號各自產生排名後加權合併，向量沒撈到但關鍵字命中極強的內容也有機會進候選池
排序品質把關	有：Cross-Encoder Reranker（bge-reranker-v2-m3, 5.68億參數）	有：Cross-Encoder Reranker（bge-reranker-base, 2.78億參數；受限於本機記憶體，是準確度與可運行性之間的取捨）
表格資料處理	全文OCR文字裡的表格未經任何結構化處理，隨頁面文字一起被貪婪字數切塊	用pdfplumber偵測表格結構、合併多列表頭、去重複列，清理成乾淨表格後再依代號分組
關鍵數值查詢	完全依賴LLM根據檢索到的context生成答案，沒有繞過LLM的查表路徑	額外提供規則式查表（SD/SR/D代號直接對照表格索引），跟AI答案雙軌互相驗證

也就是說，這不是「重做一個類似的東西」，也不是「原模組很陽春、這個專題全面碾壓」——兩邊都有向量檢索跟reranker，這部分是站在原模組的基礎上做的。真正的差異在於把「檢索到就相信」的單軌設計，換成切割更貼合文件結構、檢索管道更冗餘（向量+關鍵字雙軌）、生成後多一層事實查核的架構。以上差異的實際效果，見下方「成效驗證」的BM25分詞、清單型查詢、Groundedness三個實測案例——不是空談架構，每一項差異都有對應的具體數字佐證。

二、系統架構

系統分為「索引建立」「檢索與生成」「生成後驗證」三個階段：

索引建立：上傳 PDF → pdfplumber 解析（頁碼還原、表格結構化清理）→ 表格依牌號/代號分組逐塊切割、條文依節號/子項邊界切割 → HuggingFace Embedding 分批寫入向量庫（Chroma）+ BM25 關鍵字索引（中文逐字元分詞）+ SD/SR/D 規則式代號索引

檢索與生成：使用者問題同時做向量搜尋與 BM25 搜尋 → RRF 融合排序（向量核可閘門 + BM25 前 3 名一律放行，清單型問句額外放寬候選池與同頁名額）→ Cross-Encoder Reranker 精選 → 組成 Context → 交給 Gemini 生成回答

生成後驗證：Groundedness 檢查（答案數字是否有原文依據）+ 引用頁碼反查（頁碼是否真的被檢索到），並列出 SD/SR/D 規則式查表結果做雙軌對照

三、技術亮點

一般入門 RAG 教學通常只有「向量檢索 → LLM 生成」兩步，本專題額外做了以下強化：

環節	做法	目的
PDF 頁碼還原	逐頁偵測印刷頁碼，找不到的用鄰近錨點局部內插	附錄重新編號、章節跳頁時，引用頁碼仍正確
表格結構化清理	合併多列表頭、向下補值、去重複列，多重特徵辨識資料列	把 PDF 表格斷裂、跨頁的原始資料還原成乾淨表格
表格依代號分組切塊	依符號/牌號重複分組或純代號格式逐列切開，摘要句明講是哪個代號	避免大表格被當一個 chunk，稀釋單一代號查詢的相關性
中文 BM25 逐字分詞	英數字連續當一個 token、中文逐字拆開的正則分詞	修正中文無空白導致 BM25 全部零分、退化成雜訊的問題
混合檢索 + RRF	向量語意搜尋 + BM25 關鍵字搜尋，依排名加權融合	語意與精確關鍵字互補，比簡單聯集更可靠
向量核可閘門動態放寬	向量前 k 名 ∪ BM25 前 3 名才放行，不限代號類查詢	修正一般問句被閘門誤擋、答成查無相關規定的問題
Cross-Encoder Reranker	用 BAAI/bge-reranker-base 對候選重新精算相關性	比向量/BM25 各自獨立打分更準，是最後一道把關
生成 Prompt 措辭校準	避免過度強調「數值」，規定/程序皆可直接回答	修正程序性問題因用詞被誤判無數值可答而拒答
Groundedness 檢查	比對答案數值是否出現在原文中，濾除列表編號誤報	攔截「檢索沒命中、LLM 卻自己編數字」的幻覺
引用頁碼反查	答案引用頁碼若不在檢索範圍內，主動跳警告	防止 LLM 編造看似合理但查無來源的引用
規則式雙軌對照	正則規則直接從表格索引查 SD/SR/D 代號數據	不透過 LLM，提供獨立、可驗證的查詢路徑
索引建立分批 Embedding	冷啟動建索引時分批寫入向量庫	降低尖峰記憶體用量，避免資源有限環境下建索引被砍掉
清單型查詢兜底機制	偵測清單型問句時放寬候選池，並直接從未切塊的表格索引補上完整資料	解決分組後表格 chunk 排名不穩定、清單答案漏項的問題

四、成效驗證（量化比較）

不只是文字描述「有變好」，以下是實測過的具體案例，依效果規模排序：

BM25 中文分詞：修正前後對照

實測查詢「SD420W和SD690的抗拉強度和降伏強度」（句子沒有任何空白）：

項目	修正前	修正後
分詞結果	整句被當成唯一一個 token，跟語料庫任何文件都對不上	拆成「SD420W」「和」「SD690」等有意義的 token
BM25 前 2 名	跟問題完全無關的文件內容（分數接近雜訊）	SD420W、SD690 專屬表格 chunk（分數 27.7、27.2）

生成 Prompt 措辭：修正前後對照

實測查詢「鋼筋重驗不合格時怎麼處理」：

項目	修正前	修正後
檢索狀態	正確段落已檢索到，Reranker 分數 0.965（滿分等級）	同一份 context
最終答案	「查無相關規定，請人工確認」	正確列出 7.4 節重驗程序
原因	Prompt 反覆強調「數值」，模型誤判程序性問題為無數值可答	措辭改為數值、規定或處理程序皆可直接回答

清單型查詢候選池放寬：修正前後對照

實測查詢「竹節鋼筋的標示代號有哪些」（表格共有 13 個代號）：

項目	修正前	修正後
候選來源	分組後的單代號 table chunk 各自跟其他段落競爭候選名額	清單型問句觸發候選池放寬 + 未切塊表格索引兜底
最終答案	只列出 D10～D25，共 6 個代號	完整列出 D10～D57，全部 13 個代號
原因	FINAL_TOP_N 上限 + 分散 chunk 排名不穩定，未涵蓋的代號被擠出候選池	偵測清單型問句後直接從未切塊的表格資料補上完整內容

表格辨識覆蓋率：全面掃描前後對照

系統性掃描兩份規範全部頁面，比對「pdfplumber 有偵測到表格結構」與「清理後實際產出乾淨表格」的差異：

項目	修正前	修正後
疑似辨識失敗頁面數	24 頁	11 頁
剩餘頁面性質	部分為真正表格辨識失敗	逐頁人工確認，全部為封面／目錄／修訂歷史等本來就不是表格的內容

Groundedness 誤報率：修正前後對照

規模較小、但反映測試涵蓋到多細的邊界情況：原始版本的防幻覺檢查會把答案裡所有數字都拿去跟原文比對，但這樣連 LLM 自己排版用的列表編號也會被誤判成「查無依據的數字」：

項目	修正前	修正後
測試輸入	含列表編號的答案（如「1. 降伏強度 420MPa」「2. 抗拉強度 620MPa」「(3) 伸長率 12%」）	同一段答案
誤報數量	3 個（1、2、3 被誤判為查無原文依據）	0 個
原因	直接抓答案裡所有數字比對	比對前先濾除行首列表編號，只比對真正的數值

正式測試結果

完成本輪所有修正後，以 50 題涵蓋兩份規範的測試集正式驗證，嚴格正確率 92%（46/50），含部分正確計分則為 94%。所有失敗與部分正確案例都已定位根本原因並記錄——2 題屬於窮舉型清單或短句檢索的已知結構性限制，1 題是原文缺乏乾淨定義句可回答，1 題是 PDF 圖示轉純文字後失去空間對應關係導致的標示對調，沒有原因不明的隨機失誤。測試過程中每次程式修改都會重跑先前已通過的題目，確認修正沒有互相打壞（例如向量核可關門放寬後重測代號類查詢、生成 Prompt 校準後重測負向測試案例），完整逐題明細請見 GitHub 專案中的 TESTING.md。

五、實際運作畫面



圖 1：上傳規範 PDF、建立索引



圖 2：AI 智慧摘要——精確引用頁碼與表格編號回答

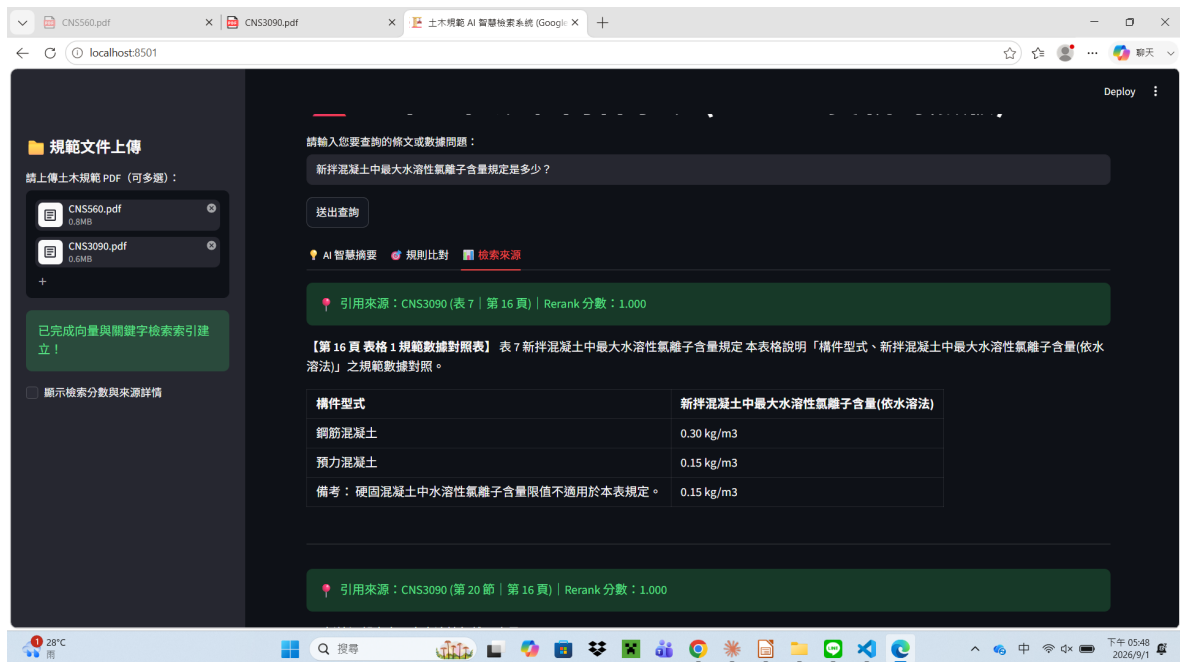


圖 3：檢索來源——可看到每筆候選的 Reranker 分數



圖 4：規則比對——SD/SR 鋼筋牌號與 D 標稱尺寸代號皆可直接查表，與 AI 答案雙軌對照

六、技術棧

類別	選用技術
前端／應用框架	Streamlit
PDF 解析	pdfplumber
向量資料庫	Chroma
Embedding 模型	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Reranker 模型	BAAI/bge-reranker-base

類別	選用技術
關鍵字檢索	BM25 (rank_bm25)
LLM	Google Gemini (透過 langchain-google-genai)
框架整合	LangChain

七、已知限制與未來規劃

模型選擇受硬體限制：Reranker 原採用效果更好的 BAAI/bge-reranker-v2-m3（5.68 億參數），但在記憶體有限的機器上會導致程序被系統強制終止，因此改用較輕量的 bge-reranker-base，是準確度與可運行性之間的取捨。

Groundedness 檢查仍屬粗略比對：目前以數字集合差集判斷，未涵蓋單位、語意層級的一致性檢查。

語料範圍：目前僅支援使用者臨時上傳的 PDF，尚無固定的規範資料庫版本管理機制。

規則式查表覆蓋範圍：目前支援 SD/SR 鋼筋牌號與 D 標稱尺寸代號兩種格式，其他規範數值（如混凝土強度等級、坍度數值區間）需要「數值範圍比對」而非「精確代號比對」，屬於不同複雜度的擴充，尚未涵蓋。

窮舉型清單問題，表格類已修正、純文字類仍有限制：問「列出所有 XX」這種需要彙整多個 chunk 才能答全的問題，若答案來自某張已被分組切塊的表格，系統會在偵測到清單型問句時動態放寬候選池並直接從未切塊的表格索引補上完整內容，不受個別 chunk 排名不穩定的影響。但若答案是分散在條文裡的純文字列舉，目前仍受限於送進 LLM 的候選數上限，可能只答出部分項目，尚待針對這類純文字清單設計對應的兜底機制。

短句 chunk 對通用字查詢的命中率較低：中文 BM25 採逐字元分詞，若條文本身很短且查詢只共用單一常見字（而非多字詞組），字元層級的區分度不夠，可能檢索不到——這是逐字元分詞方法的結構性限制。

PDF 圖示型標示範例經純文字擷取後會失去空間對應關係：pdfplumber 的文字擷取只保留閱讀順序，不保留座標位置，遇到「一排代號＋下方對應標籤」這種圖示型排版，標籤與代號的對應關係在純文字裡會變得模糊，LLM 有時會猜錯配對方向，這是 PDF 純文字擷取先天的限制，不是檢索邏輯的問題。

完整原始碼、README 與更多截圖請見 GitHub：<https://github.com/ching-pochiu/rag>